

Database	Primary strength	Deployment	Supported features	Community/framework support
Weaviate	Cloud-native, scalability	Docker, Kubernetes, Cloud	Hybrid search, RAG, Metadata filtering	LangChain, LlamaIndex, Go, Python, Java
Qdrant	High performance, speed	Docker, Cloud	Hybrid search, Multi-vector, Quantization	LangChain, LlamaIndex, Python, Rust, Go
Milvus	Scalability, large datasets	Docker, Kubernetes, Cloud	Hybrid search, Multi-vector, Replication	LangChain, LlamaIndex, Python, Java, Go
pgvector	Integration, simplicity	PostgreSQL Extension	L distance, Inner product, Cosine similarity	LangChain, LlamaIndex, Python, Go, Rust

Table 1: A comparison of FOSS vector databases

The foundation: Selecting your FOSS vector database

Building a production-ready RAG system requires a robust and scalable vector database. These systems are designed to manage vast volumes of unstructured data by storing and indexing high-dimensional vector embeddings. The true power of these databases for enterprise applications lies in their ability to perform semantic searches, which retrieve information based on conceptual similarity rather than just keyword matches. Beyond basic search, enterprise-class vector databases offer crucial features such as metadata filtering and scalability to handle growing datasets without compromising performance.

While commercial solutions like Pinecone, Weaviate, and Chroma offer managed services, a vibrant ecosystem of open source alternatives provides compelling self-hosted options. This ecosystem is not a monolith; each solution possesses a distinct architectural philosophy and a unique set of strengths, making the choice a strategic decision based on the specific needs of a project.

Weaviate: As a cloud-native, open source vector database, Weaviate is designed to store both objects and vectors, seamlessly combining vector search with structured filtering. Its architecture is built in Go for speed and reliability, and it offers native support for horizontal scaling, making it a strong choice for mission-critical applications. Its ease of deployment via Docker and Kubernetes and its robust Python client make it highly accessible for developers prototyping locally or deploying at scale.

Qdrant: Known for its high performance, Qdrant leads in requests-per-second and low latency, with response times as low as milliseconds for millions of vectors. Its performance-first design makes it an ideal choice for applications where speed is paramount, such as real-time recommendation engines or high-traffic search APIs. Qdrant also offers built-in features like vector compression and multi-vector support, which can significantly reduce memory usage and enhance search experiences.

Milvus: Built for large-scale distributed systems, Milvus is a high-performance vector database that excels at managing massive datasets for generative AI applications. Its architecture is designed to handle immense data scales, making it a go-to solution for companies with petabytes of unstructured data. For local development and smaller projects, the Milvus Lite option provides a convenient, file-based solution that doesn't require a full server setup, catering directly to the needs of individual developers and small teams. Milvus's strong integration with Docker makes it easy for Linux administrators to get started.

The pgvector extension: For organisations already invested in the PostgreSQL ecosystem, pgvector offers a simple and powerful alternative. Instead of deploying and managing a separate database system, pgvector allows for the storage and retrieval of vector embeddings directly within a PostgreSQL instance. This option appeals to administrators who value integrated, battle-tested tooling and a single-database approach for their applications.

Table 1 provides a comparison to help guide the decision-making process based on specific project requirements.

The blueprint: The RAG system architecture

An enterprise RAG system is more than just a vector database; it is a meticulously engineered pipeline that orchestrates the flow of data from raw documents to a contextualized AI response. The process can be broken down into a series of core stages, each critical to the system's overall performance and accuracy. The success of this pipeline is acutely dependent on the quality of data preparation and ingestion, a foundational step that is often underestimated. A technically advanced vector database cannot compensate for a flawed ingestion process.

Ingestion and chunking: The first step involves preparing the source data. Unstructured documents, such as PDFs or web pages, must be broken down into smaller, logical pieces, or 'chunks', for the LLM to process. This process is crucial because a chunk that is too large can contain irrelevant information, while a chunk that is too small can lose essential